

GameRent

Relatório Técnico de Sistemas Distribuídos — Entrega 1
Plataforma de Aluguel de Jogos de Tabuleiro na AWS

Integrantes do Grupo:

Maria Eduarda (Front-end, Infraestrutura, Segurança & Resiliência)

Luca Samuel (Backend, Coordenação, Consistência & Dados)

Kauan Simão (Middleware, APIs, Comunicação & Integração)

Disciplina: Sistemas Distribuídos

Provedor de Nuvem: Amazon Web Services (AWS)

1. Escolha e Descrição do Tema

O projeto **GameRent** consiste em uma plataforma de software voltada para o aluguel compartilhado de jogos de tabuleiro, operando como um ecossistema digital que conecta locadores (proprietários de jogos) e locatários de maneira eficiente e geograficamente localizada. O modelo de negócios é estruturado para funcionar de forma semelhante ao iFood: embora o objetivo seja facilitar transações entre usuários finais (peer-to-peer em essência), a **arquitetura de rede adota o modelo clássico Cliente-Servidor com Intermediário (Broker/API Gateway)**. Sob essa abordagem, os clientes (dispositivos móveis e navegadores web) nunca interagem de forma direta (peer-to-peer na rede) para trocar dados sensíveis, negociar ou processar transações. Toda a comunicação é rigidamente centralizada, filtrada e orquestrada pela plataforma hospedada na Amazon Web Services (AWS).

Justificativas Técnicas do Modelo Distribuído

A escolha de uma arquitetura estritamente cliente-servidor com um middleware intermediador centralizador se sustenta sobre duas premissas fundamentais de engenharia de software e sistemas distribuídos:

- **Garantia de Privacidade de Geolocalização Relativa:** Para assegurar a integridade física e a privacidade dos usuários, a localização real (coordenadas GPS exatas) dos locadores nunca é exposta na rede aos locatários. Quando um locatário realiza uma busca, suas coordenadas são enviadas ao Servidor Central (Backend). O backend realiza os cálculos espaciais em memória diretamente no servidor e retorna ao cliente apenas a distância linear calculada (ex: 'Jogo disponível a 1.2 km de distância'). Isso impossibilita que terceiros interceptem ou façam engenharia reversa para descobrir o endereço exato dos usuários.
- **Coordenação Centralizada contra Reservas Concorrentes (Double-Booking):** Em um sistema descentralizado P2P puro, garantir a exclusão mútua e a consistência forte em transações concorrentes é um problema de alta latência e complexidade. Com o modelo cliente-servidor intermediado, o servidor central age como um árbitro único e confiável de estado. Se dois usuários tentarem reservar o mesmo exemplar físico de um jogo de tabuleiro simultaneamente, o servidor processará as requisições em ordem serial de chegada no banco de dados, aplicando exclusão mútua distribuída para confirmar uma transação e rejeitar (ou enfileirar) a outra imediatamente.

2. Diagrama Inicial de Arquitetura (Alto Nível)

A arquitetura proposta foi desenhada para isolar as pontas de consumo (Clientes) dos serviços de processamento interno e armazenamento. O diagrama de alto nível a seguir descreve o fluxo de comunicação ponta a ponta na nuvem AWS, demonstrando a centralização das requisições no middleware de entrada e o processamento de regras lógicas de geolocalização e consistência.

GameRent - Arquitetura Inicial de Alto Nível

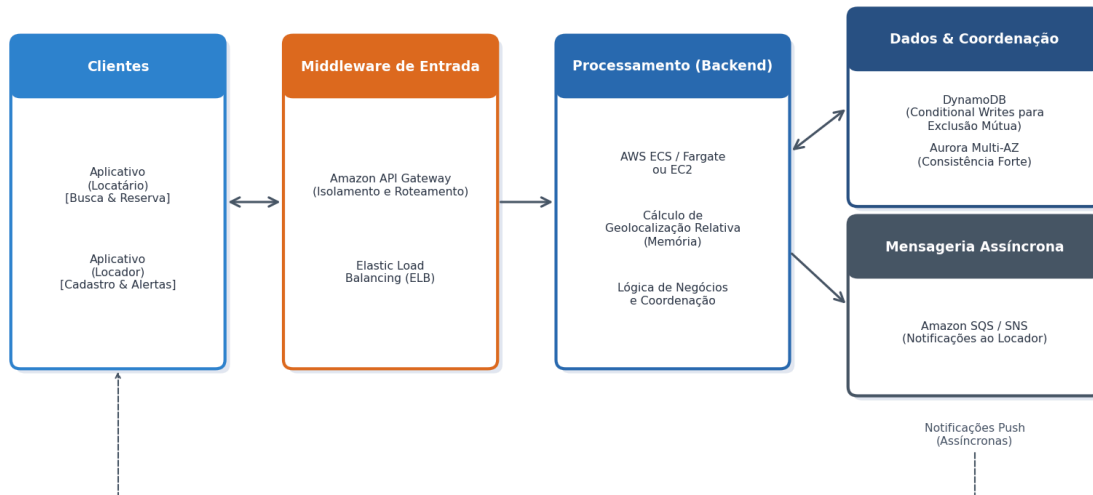


Figura 1: Topologia de Arquitetura Cliente-Servidor do GameRent na Nuvem AWS

Como visualizado no diagrama, a interação ocorre em camadas lógicas bem estruturadas:

- **Canal de Entrada Único:** Os clientes (Locatário e Locador) disparam chamadas REST ou gRPC que batem inicialmente no Elastic Load Balancer (ELB) e no Amazon API Gateway. Nenhum outro recurso interno de processamento possui endereço IP público, o que cria uma barreira de segurança impenetrável.
- **Processamento Backend Elástico:** As instâncias hospedadas em AWS ECS Fargate ou EC2 processam a lógica pesada. A localização geográfica dos usuários é calculada na memória do servidor físico do container e as chamadas de banco de dados são executadas de forma enclausurada em uma rede privada virtual (VPC).
- **Persistência e Mensageria Assíncrona:** Para manter a alta disponibilidade e mitigar o acoplamento temporal, o backend delega ações de notificação para filas assíncronas (Amazon SQS). Se um aluguel for efetuado, o SQS/SNS se encarrega de empurrar o alerta de confirmação ao dispositivo do locador sem travar a thread de processamento do locatário.

3. Quadro de Seleção de Tecnologias

Abaixo está o detalhamento e preenchimento técnico do quadro de seleção de tecnologias distribuídas exigido pela disciplina, com o mapeamento direto para os serviços oferecidos no ecossistema AWS e suas respectivas justificativas no GameRent:

Categoria / Camada	Função no GameRent	Tecnologia AWS & Justificativa Técnica
Middleware de Entrada	Ponto único de contato público, recebendo requisições, aplicando roteamento seguro e distribuindo a carga de requisições de buscas.	AWS API Gateway e Elastic Load Balancing (ELB). Isolam as redes privadas internas e expõem endpoints REST/gRPC limpos aos clientes web/mobile.
Processos e Virtualização	Hospedagem física, execução elástica e computação da regra de negócio central de geolocalização e reservas.	AWS ECS / Fargate ou instâncias EC2. Containers Fargate garantem uma execução serverless sem necessidade de gerenciar o sistema operacional subjacente.
Comunicação	Orquestração de chamadas de rede síncronas entre cliente e servidor, além do disparo de eventos assíncronos (notificações de aluguel).	APIs REST/gRPC e Amazon SQS / SNS. O SQS desacopla o envio de notificações de confirmação de aluguel enviadas ao locador, evitando gargalos de concorrência.
Coordenação	Prevenção e controle de concorrência distribuída extrema, garantindo exclusão mútua e impedindo locações simultâneas de um único item.	DynamoDB Conditional Writes. Gravações condicionais no banco de dados não-relacional servem como semáforos distribuídos rápidos e nativos.
Nomeação	Descoberta de microserviços dinâmicos e resolução de nomes internos na infraestrutura interna na nuvem.	AWS Cloud Map e Amazon Route 53. Permitem mapear e registrar dinamicamente os IPs e portas de containers do ECS, resolvendo domínios privados.
Consistência e Replicação	Persistência confiável de transações financeiras, cadastros e catálogo com alta resiliência multi-zona geográfica.	Amazon Aurora Multi-AZ ou DynamoDB. O Aurora fornece replicação de dados síncrona automática entre diferentes zonas de disponibilidade físicas da AWS.
Tolerância a Falhas	Monitoramento preventivo de anomalias no sistema, escalabilidade	Auto Scaling Groups e Amazon CloudWatch. Se uma instância do ECS

	sob demanda e recuperação rápida em falhas de servidores.	ou EC2 falhar, o Auto Scaling destrói e cria um novo nó automaticamente em segundos.
Segurança	Gerenciamento fino de controle de acesso (IAM), isolamento de credenciais de banco e criptografia de dados em repouso e trânsito.	AWS IAM, Secrets Manager e KMS. Permite autenticação estrita baseada no princípio do menor privilégio e guarda encriptada de senhas de produção.

4. Distribuição de Responsabilidades

A divisão do trabalho prático foi estruturada meticulosamente para garantir que cada participante lidere frentes de desenvolvimento completamente alinhadas com as diretrizes e módulos técnicos avaliados na disciplina de Sistemas Distribuídos:

Maria Eduarda — Infraestrutura, Segurança, Resiliência & Front-end

Responsável por conceber a base arquitetural resiliente onde o sistema rodará e a interface que os clientes utilizam para consumir o backend.

- **Processos e Virtualização:** Configuração da execução de microsserviços em servidores virtuais EC2 ou containers autogerenciados no AWS ECS/Fargate.
- **Descoberta de Serviços e Segurança:** Mapeamento do Route 53 e Cloud Map, além de garantir segurança rígida utilizando AWS IAM, Secrets Manager e chaves KMS.
- **Tolerância a Falhas:** Programação de Auto Scaling Groups e alarmes no CloudWatch para resposta rápida diante de gargalos de rede ou quebras de nós.
- **Desenvolvimento Front-end:** Liderança na construção do aplicativo e da interface web que servem como ponto de partida das requisições.

Luca Samuel — Backend, Coordenação, Consistência & Dados

Responsável pelo núcleo de processamento lógico distribuído, garantia da consistência dos dados e controle transacional rígido no servidor central.

- **Algoritmos de Sincronização e Coordenação:** Implementação de semáforos distribuídos e exclusão mútua de reservas usando conditional writes do DynamoDB no backend.
- **Consistência de Estado:** Modelagem de tabelas de dados no Aurora Multi-AZ e DynamoDB, validando modelos de consistência forte e replicação síncrona.
- **Lógica de Geolocalização Segura:** Desenho e codificação do algoritmo em memória no backend que calcula a distância relativa protegendo a localização de origem.

Kauan Simão — Middleware, APIs, Comunicação & Integração

Responsável por estruturar a ponte de rede síncrona e assíncrona que interliga as telas de usuário ao backend central da plataforma.

- **Configuração do Middleware de Entrada:** Implementação e tunagem das regras de roteamento e vazão de tráfego usando Amazon API Gateway e Elastic Load Balancer (ELB).
- **Comunicação RPC e Mensageria:** Padronização dos contratos de comunicação usando APIs REST/gRPC e configuração estruturada das filas assíncronas do AWS SQS/SNS.
- **Contratos de Integração:** Garantir que as buscas do catálogo do aplicativo e os pacotes de reserva transitem entre cliente e servidor com latência e pacotes otimizados.